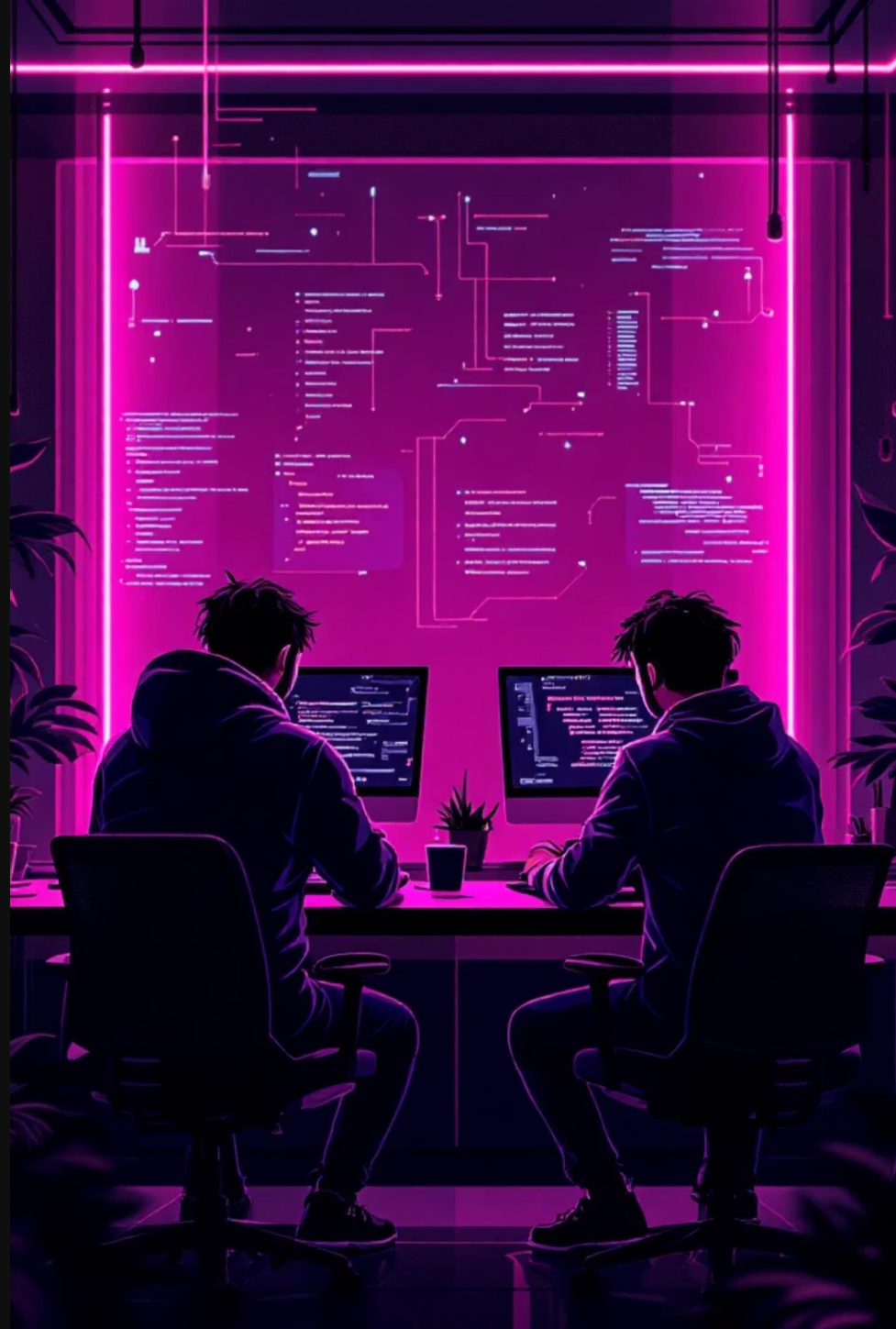
DevArena: Real-Time Competitive Coding Platform

A full-stack platform revolutionizing coding practice through live 1v1 battles, designed to transform how developers train, compete, and improve their skills in real-time competitive environments.



Why DevArena: The Problem with Current Platforms

Existing Limitations

Traditional coding platforms like LeetCode, HackerRank, and Codeforces operate in isolation. Users solve problems alone, submit solutions, and receive results without the pressure of competition or immediate feedback from opponents.

The passive learning model fails to simulate real-world technical interviews or high-pressure coding scenarios where speed, accuracy, and strategic thinking are tested simultaneously.

What DevArena Solves

Live 1v1 battles create authentic competitive pressure, forcing developers to think faster and code smarter under time constraints. Real-time opponent activity reveals strategic patterns and forces adaptive problem-solving.

The platform combines algorithmic challenge with psychological intensity, creating an experience that mirrors actual technical interviews and competitive programming contests while building confidence through repeated exposure.

Made with GAMMA

From User Login to Victory: The Complete Journey

1

Authentication

User creates account or logs in, profile data loads instantly with rating and stats

2

Dashboard

View ranking, recent matches, and click "Find Match" to enter queue

3

Matchmaking

System searches for opponent with similar skill level, creates private battle room

4

Battle Room

Both users receive same problem, timer starts, real-time coding begins

5

Submission

First to solve correctly wins, rating updates reflect skill change

Made with GAMMA

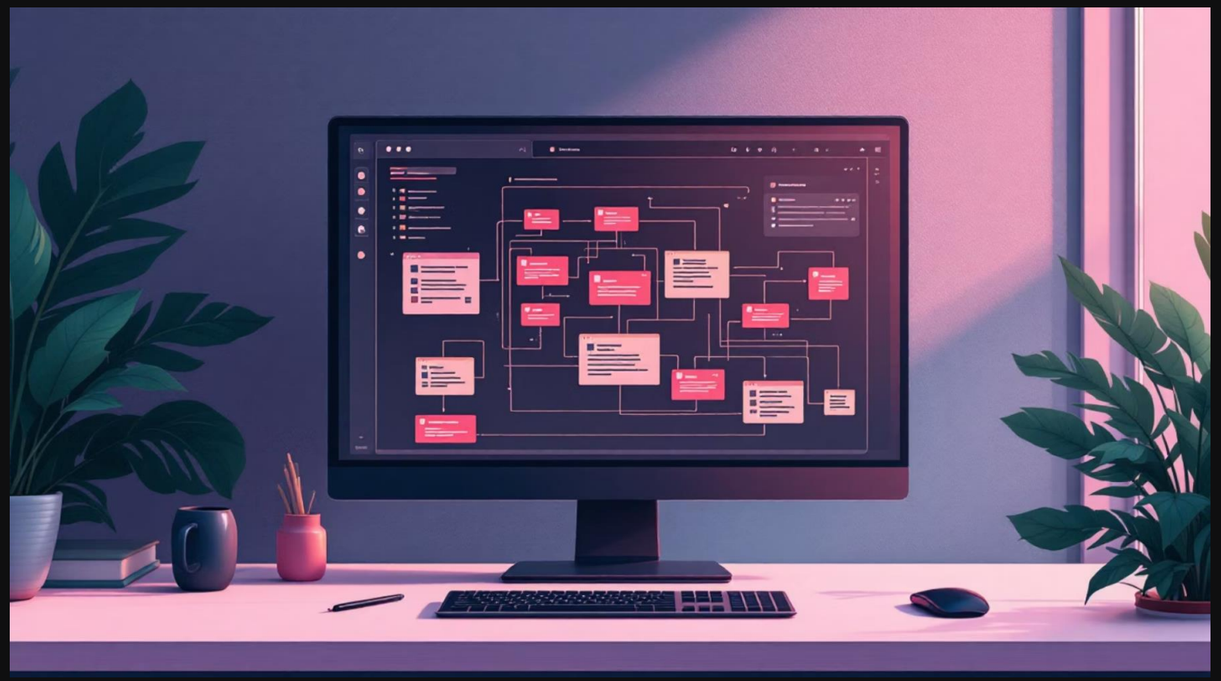
Frontend Architecture: React Component Structure

Page Structure

- Landing Page: Hero section with value proposition and CTA
- Auth Pages: Login and registration with form validation
- Dashboard: User stats, ranking, match history, and queue button
- Battle Room: Problem panel, code editor, timer, opponent activity
- Leaderboard: Global rankings with search and filter options

State Management

Context API manages user authentication state globally. Battle context handles real-time game state including timer, opponent progress, and submission results. Local component state manages editor content and form inputs.



Real-Time Updates

Socket.io client maintains persistent connection to backend, listening for events like opponent join, timer sync, code updates, and match results. Components re-render efficiently using React's reconciliation algorithm.

UI/UX Design

Clean, minimal interface prioritizes the coding experience. Dark theme reduces eye strain during extended sessions. Responsive design ensures mobile compatibility for on-the-go practice.

Made with GAMMA

Battle Room Interface: Deep Technical Breakdown

Problem Panel

Displays problem description, constraints, test cases, and difficulty rating. Collapsible sections organize information hierarchically. Syntax-highlighted examples demonstrate expected input/output formats.

Monaco Editor

Same editor powering VS Code provides intelligent code completion, syntax highlighting, bracket matching, and error detection. Supports multiple languages including JavaScript, Python, and Java with language selector dropdown.

Countdown Timer

Visual countdown creates urgency and simulates interview conditions. Timer synchronizes across both clients using server time to prevent manipulation. Warning animations trigger at 30 seconds remaining.

Activity Monitor

Real-time visualization shows opponent's typing activity, submission attempts, and progress milestones. Creates psychological pressure while revealing strategic patterns like early submissions or methodical debugging.

Submission System

Code sends to backend for validation and execution. Loading state prevents duplicate submissions. Results display test case outcomes, execution time, and memory usage metrics for post-match analysis.

Results Screen

Victory or defeat notification with comparative statistics. Highlights key differences like execution speed and code efficiency. Updates rating immediately with animation showing change magnitude.

Made with GAMMA

Backend Architecture: Node.js & Express Foundation

API Structure

RESTful endpoints organized by resource type with consistent naming conventions. Routes include /auth for authentication, /users for profile management, /matches for battle operations, and /problems for coding challenges.

Request validation middleware sanitizes inputs and prevents injection attacks. Rate limiting protects against abuse while maintaining legitimate user access during peak traffic.

Authentication Layer

JWT tokens provide stateless authentication with encrypted payloads containing user ID, username, and expiration timestamp. Refresh tokens enable session persistence without storing session data server-side.

Password hashing uses bcrypt with salt rounds to prevent rainbow table attacks. Secure cookies with HttpOnly flag prevent XSS exploitation while maintaining session state.

01

Matchmaking Logic

Queue system pairs users by skill rating within configurable tolerance ranges

02

Game State Management

Tracks active battles, problem assignments, timer status, and submission validity

03

Socket Integration

Real-time events broadcast opponent actions and sync game state across clients

Made with GAMMA

Matchmaking System: Intelligent Pairing Logic

1. **User Queues**
   Player enters matchmaking queue with current rating and selected difficulty range
2. **Search Algorithm**
   System searches for compatible opponent using skill-based matching with tolerance window
3. **Room Creation**
   Private battle room instantiated with randomized problem from appropriate difficulty tier
4. **Client Connection**
   Both users receive room credentials and connect via WebSocket for real-time updates

Scalability Considerations

Current implementation uses in-memory queue for simplicity. Production-ready version implements Redis for distributed queue management across multiple server instances. Skill-based matchmaking employs ELO rating system that adjusts pairing windows based on rating volatility and queue wait times.

Made with GAMMA

Real-Time System: Socket.io Event Flow

Connection Lifecycle

WebSocket connection establishes during page load, maintaining persistent TCP connection. Reconnection logic automatically restores connection if network drops, preventing match abandonment from transient issues.

Authentication handshake validates JWT token before accepting socket connection, ensuring only authorized users access battle rooms. Connection cleanup automatically frees resources when users disconnect.

User Joins

Client emits 'join' event with authentication token and desired difficulty

Room Created

Server creates battle room, assigns problem, broadcasts 'room ready' to both clients

Timer Sync

Server emits 'start timer' event with synchronized start time across all clients

Live Updates

Typing indicators and cursor positions broadcast continuously as users code

Submission

Code sent to execution pipeline, results broadcast to both players simultaneously

Results

Victory screen displays with comparative statistics and rating adjustments

Made with GAMMA

Code Execution Pipeline & Data Architecture

Execution Flow

01

Client Sends Code

Code and language specification packaged in JSON request

02

Backend Validation

Input sanitization prevents malicious code execution

03

Judge0 API

External execution engine runs code in isolated container

04

Result Processing

Execution time, memory usage, and test outcomes analyzed

05

Response Broadcast

Results sent to both clients with victory determination

Database Collections

ELO Rating System

Standard ELO algorithm adjusts ratings based on expected versus actual outcomes. Higher-rated opponents yield larger point gains from victories. System prevents rating inflation through dynamic K-factor adjustment based on match count and rating stability.

Users Collection

User profiles store authentication data, rating history, match statistics, and preferences with unique identifiers and encrypted passwords.

Matches Collection

Match records include timestamps, participants, problem ID, winner, and submission details for analytics and replay functionality.

Problems Collection

Problem library contains descriptions, test cases, difficulty ratings, and solution statistics for balanced matchmaking.

Made with GAMMA

Deployment Architecture & Advanced Features

Production Stack

- Frontend: Vercel for CDN distribution and edge network optimization
- Backend: Render or Railway for Node.js server with auto-scaling
- Database: MongoDB Atlas for managed NoSQL with global replication
- Environment: Separate configurations for development, staging, and production

Security Measures

Environment variables store secrets using encrypted configuration. HTTPS enforcement with HSTS headers prevents downgrade attacks. Input validation middleware sanitizes all user-provided data before database persistence.

Scalability Strategy

Horizontal scaling adds server instances behind load balancer. Redis manages WebSocket connections for distributed messaging. Database sharding partitions data by user regions for localized access patterns.

Advanced Features

- **Spectator Mode**
  Watch live battles with commentary overlay and analysis tools
- **Tournament System**
  Bracket-style competitions with automated scheduling and prizes
- **Live Chat**
  Real-time messaging during matches with moderation filters
- **AI Problem Curation**
  Machine learning suggests personalized challenges based on skill gaps

Portfolio Impact: DevArena demonstrates full-stack proficiency across modern technologies while solving real competitive programming challenges. The platform showcases ability to architect scalable systems, implement real-time features, and deliver polished user experiences—making it an exceptional portfolio centerpiece for technical interviews and startup opportunities.

Made with GAMMA